

ovs的classifier创建和查询

简介

classifier的原理

table 74的classifier构造

cls→subtables_map和cls→subtables的构造

subtable→rules的插入流表

subtable→indices的构造

openflow流表插入到classifier的流程

table 74的rule查找

简介

在前一篇文档中：[revalidator线程删除mask不一致datapath flow分析](#)，分析了revalidator线程通过比较xlate mask和dp mask，发现mask不一致，而删除dp flow的原因。这里进一步分析为什么xlate mask中会多出 `nd_target` 和 `nd_ttl` 字段。

我们将环境中的相关openflow流表，简要整理了一下，可以通过以下的testcase进行模拟，这里我们将table 73和table 74的openflow流表从环境中全部拿出来了，对于table 0和table 8,只使用了有用的table部分。

```
1 AT_SETUP([flow classifier - ipv6 ND dependency test])
2 OVS_VSWITCHD_START
3 add_of_ports br0 1 2
4 AT_DATA([flows.txt], [dn1
5 table=0, in_port=1,priority=100 actions=load:0xd->NXM_NX_REG13[[0..15]
6 table=8, priority=50,metadata=0x4 actions=load:0->NXM_NX_REG10[[12]],r
7 table=73, priority=95,arp,reg14=0x5,metadata=0x4 actions=resubmit(,74)
8 table=73, priority=95,arp,reg14=0x4,metadata=0x4 actions=resubmit(,74)
9 table=73, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:
10 table=73, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:
11 table=73, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:
12 table=73, priority=90,ipv6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:b
13 table=73, priority=90,ipv6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:b
14 table=73, priority=90,ip,reg14=0x4,metadata=0x4,dl_src=fa:16:3e:14:72:
15 table=73, priority=90,udp,reg14=0x4,metadata=0x4,dl_src=fa:16:3e:14:72:
16 table=73, priority=80,reg14=0x5,metadata=0x4 actions=load:0x1->NXM_NX_
17 table=73, priority=80,reg14=0x4,metadata=0x4 actions=load:0x1->NXM_NX_
18
19 table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_typ
20 table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_typ
21 table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_typ
22 table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_typ
23
24 table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:
25 table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:
26
27 table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_typ
28 table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_typ
29
30 table=74, priority=90,arp,reg14=0x4,metadata=0x4,dl_src=fa:16:3e:14:72:
31
32 table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_typ
33 table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_typ
34
35 table=74, priority=80,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_typ
36 table=74, priority=80,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_typ
37 table=74, priority=80,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_typ
38 table=74, priority=80,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_typ
39
40 table=74, priority=80,arp,reg14=0x4,metadata=0x4 actions=load:0x1->NXM
41 table=74, priority=80,arp,reg14=0x5,metadata=0x4 actions=load:0x1->NXM
```

```

42  })
43  ovs-ofctl add-flows br0 flows.txt
44
45  AT_CHECK([ovs-appctl ofproto/trace br0 "in_port=1,eth_src=fa:16:3e:ce:b
46  AT_CHECK([tail -2 stdout], [0],
47  [Megaflow: recirc_id=0,eth,icmp6,in_port=1,dl_src=fa:16:3e:ce:bd:06,i
48  Datapath actions: 2
49  ])
50  OVS_VSWITCHD_STOP
51  AT_CLEANUP

```

这里ofproto/trace的结果如下：

```

1  Flow: icmp6,in_port=1,vlan_tci=0x0000,dl_src=fa:16:3e:ce:bd:06,dl_dst=f
2
3  bridge("br0")
4  -----
5  0. in_port=1, priority 100
6     load:0xd->NXM_NX_REG13[0..15]
7     load:0xb->NXM_NX_REG11[]
8     load:0xa->NXM_NX_REG12[]
9     load:0x4->OXM_OF_METADATA[]
10    load:0x5->NXM_NX_REG14[]
11    load:0->NXM_NX_REG13[16..31]
12    resubmit(,8)
13  8. metadata=0x4, priority 50
14     load:0->NXM_NX_REG10[12]
15     resubmit(,73)
16  73. ipv6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:bd:06,ipv6_src=f
17     resubmit(,74)
18     74. No match.
19         drop
20     output:2
21
22  Final flow: icmp6,reg11=0xb,reg12=0xa,reg13=0xd,reg14=0x5,metadata=0x4,
23  Megaflow: recirc_id=0,eth,icmp6,in_port=1,dl_src=fa:16:3e:ce:bd:06,ipv6
24  Datapath actions: 2

```

这里我们主要分析classifier查询了table 74的openflow流表后，wc如何被设置了 `nd_target` 和 `nd_ttl` 字段。

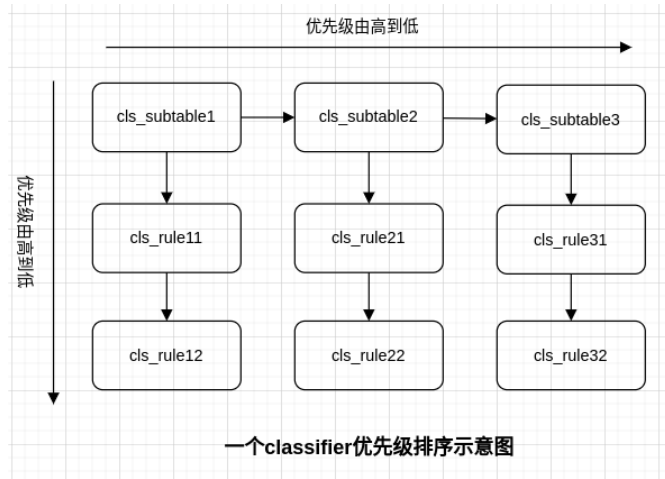
classifier的原理

ovs的classifier主要用于openflow流分类，属于match+action中的match。ofproto-dpif-xlate用于将openflow的action翻译成datapath的action。

对于openflow流表，对于classifier有如下关系：

- 一个openflow table对应一个classifier.
- 一个classifier下包含多个cls_subtables.
- 一个cls_subtable下包含多个cls_rules.

优先级排序示意图如下：



- 具有相同mask掩码flow的cls_rule放到同一个cls_subtable下。
- 具有相同 **flow & mask** 分段掩码累计值的flow的cls_rule放到同一个cls_subtable→rules下。
- 同一个cls_subtable下的cls_rules按照优先级由高到低进行排序存储。
- cls_subtables也是按照优先级由高到低进行排序存储，其中cls_subtable的优先级是cls_rules中的最高优先级，即cls_subtable1的优先级为cls_rule11的优先级。

分段示意图如下：



```

1 struct flow {
2     /* Metadata */
3     struct flow_tnl tunnel; /* Encapsulating tunnel parameters. */
4     ovs_be64 metadata; /* OpenFlow Metadata. */
5     uint32_t regs[FLOW_N_REGS]; /* Registers. */
6     uint32_t skb_priority; /* Packet priority for QoS. */
7     uint32_t pkt_mark; /* Packet mark. */
8     uint32_t dp_hash; /* Datapath computed hash value. The ex
9                        * computation is opaque to the user sp
10
11     union flow_in_port in_port; /* Input port.*/
12     uint32_t recirc_id; /* Must be exact match. */
13     uint8_t ct_state; /* Connection tracking state. */
14     uint8_t ct_nw_proto; /* CT orig tuple IP protocol. */
15     uint16_t ct_zone; /* Connection tracking zone. */
16     uint32_t ct_mark; /* Connection mark.*/
17     ovs_be32 packet_type; /* OpenFlow packet type. */
18     ovs_u128 ct_label; /* Connection label. */
19     uint32_t conj_id; /* Conjunction ID. */
20     ofp_port_t actset_output; /* Output port in action set. */
21
22     /* L2, Order the same as in the Ethernet header! (64-bit aligned) */
23     struct eth_addr dl_dst; /* Ethernet destination address. */
24     struct eth_addr dl_src; /* Ethernet source address. */
25     ovs_be16 dl_type; /* Ethernet frame type.
26                        * Note: This also holds the Ethertype
27                        * packets of type PACKET_TYPE(1, Ether
28
29     uint8_t pad1[2]; /* Pad to 64 bits. */
30     union flow_vlan_hdr vlans[FLOW_MAX_VLAN_HEADERS]; /* VLANs */
31     ovs_be32 mpls_lse[ROUND_UP(FLOW_MAX_MPLS_LABELS, 2)]; /* MPLS label
32                                                            (with padd
33
34     /* L3 (64-bit aligned) */
35     ovs_be32 nw_src; /* IPv4 source address or ARP SPA. */
36     ovs_be32 nw_dst; /* IPv4 destination address or ARP TPA.
37     ovs_be32 ct_nw_src; /* CT orig tuple IPv4 source address. */

```

```

35     ovs_be32 ct_nw_dst;          /* CT orig tuple IPv4 destination address. */
36     struct in6_addr ipv6_src;    /* IPv6 source address. */
37     struct in6_addr ipv6_dst;    /* IPv6 destination address. */
38     struct in6_addr ct_ipv6_src; /* CT orig tuple IPv6 source address. */
39     struct in6_addr ct_ipv6_dst; /* CT orig tuple IPv6 destination address. */
40     ovs_be32 ipv6_label;         /* IPv6 flow label. */
41     uint8_t nw_frag;             /* FLOW_FRAG_* flags. */
42     uint8_t nw_tos;              /* IP ToS (including DSCP and ECN). */
43     uint8_t nw_ttl;              /* IP TTL/Hop Limit. */
44     uint8_t nw_proto;            /* IP protocol or low 8 bits of ARP opcode. */
45     /* L4 (64-bit aligned) */
46     struct in6_addr nd_target;   /* IPv6 neighbor discovery (ND) target. */
47     struct eth_addr arp_sha;     /* ARP/ND source hardware address. */
48     struct eth_addr arp_tha;     /* ARP/ND target hardware address. */
49     ovs_be16 tcp_flags;          /* TCP flags/ICMPv6 ND options type. */
50     ovs_be16 pad2;               /* Pad to 64 bits. */
51     struct ovs_key_nsh nsh;      /* Network Service Header keys */
52
53     ovs_be16 tp_src;             /* TCP/UDP/SCTP source port/ICMP type. */
54     ovs_be16 tp_dst;             /* TCP/UDP/SCTP destination port/ICMP code. */
55     ovs_be16 ct_tp_src;          /* CT original tuple source port/ICMP type. */
56     ovs_be16 ct_tp_dst;          /* CT original tuple dst port/ICMP code. */
57     ovs_be32 igmp_group_ip4;     /* IGMP group IPv4 address/ICMPv6 ND reason. */
58     /* field. */
59     /* Keep last for BUILD_ASSERT_DECL below. */
60     ovs_be32 pad3;              /* Pad to 64 bits. */
61 };

```

table 74的classifier构造

subtable的数据结构如下：

```

1  /* Classifier internal definitions, subject to change at any time. */
2  /* 分类器内部定义的数据结构 */
3  /* A set of rules that all have the same fields wildcarded. */
4  struct cls_subtable {
5      struct cmap_node cmap_node; /* Within classifier's 'subtables_map'. */
6
7      /* These fields are only used by writers. 这些域只能被写者改写 */
8      int max_priority;            /* Max priority of any rule in subtable. */
9      unsigned int max_count;      /* Count of max_priority rules. 最大个数 */
10
11     /* Accessed by iterators. 使用迭代器遍历访问同一掩码的规则链表 */
12     struct rculist rules_list;    /* Unordered. 无序的规则链表 */
13
14     /* Identical, but lower priority rules are not inserted to any of the
15      * following data structures. */
16
17     /* These fields are accessed by readers who care about wildcarding. */
18     const uint8_t n_indices;      /* How many indices to use for wildcarding. */
19     //这里CLS_MAX_INDICES + 1是因为有四个段，最后一个段单独处理。
20     const struct flowmap index_maps[CLS_MAX_INDICES + 1]; /* Stage maps. */
21     //前缀树的前缀长度，所有前缀有相同的前缀长度。
22     unsigned int trie_plen[CLS_MAX_TRIES]; /* Trie prefix length in 'mask'. */
23     /* (runtime configurable). */
24     const int ports_mask_len;
25     //阶段查询hash map，保存的是前三个段的hash值，可以通过hash进行快速过滤
26     struct cmap indices[CLS_MAX_INDICES]; /* Staged lookup indices. 掩码 */
27     rcu_trie_ptr ports_trie;        /* NULL if none. 前缀树，基于掩码 */
28
29     /* These fields are accessed by all readers. */
30     /* 根据最后一个段进行hash，确定hash桶，然后进行匹配 */
31     struct cmap rules;             /* Contains 'cls_match'es. */
32     const struct minimask mask;    /* Wildcards for fields. 掩码 */
33     /* 'mask' must be the last field. */
34 };

```

这里我们不关注 Trie 的部分。

这里table 74是一个classifier，根据mask的hash值作为key，新建的subtable为value来保存到 **cls->subtables_map** 中。所以这里的cls→subtables_map值如下：

subtable名称(假设的)	Hash 值	OpenFlow 流表规则
subtable1	0xfba639d7	table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=fa:16:3e:ce:bd:06 actions=load:0->NXM_NX_REG10[12] table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12] table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=fa:16:3e:14:72:eb actions=load:0->NXM_NX_REG10[12] table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12]
subtable2	0x26716a2b	table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:bd:06,icmp_type=136,icmp_code=0,nd_target=fd1f:9e62:3182:608c::a2,nd_ttl=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12] table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:bd:06,icmp_type=136,icmp_cod

		e=0,nd_target=fd1f:9e62:3182:608c::a2,nd_ttl=fa:16:3e:ce:bd:06 actions=load:0->NXM_NX_REG10[12]
subtable3	0x1e6f068f	table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_type=136,icmp_code=0,nd_ttl=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12] table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_type=136,icmp_code=0,nd_ttl=fa:16:3e:14:72:eb actions=load:0->NXM_NX_REG10[12]
subtable4	0x60f3eccd	table=74, priority=90,arp,reg14=0x4,metadata=0x4,dl_src=fa:16:3e:14:72:eb,arp_spa=192.168.11.90,arp_sha=fa:16:3e:14:72:eb actions=load:0->NXM_NX_REG10[12]
subtable5	0x3f25d2cc	table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_type=136,icmp_code=0,nd_target=fe80::f816:3eff:fece:bd06,nd_ttl=fa:16:3e:ce:bd:06 actions=load:0->NXM_NX_REG10[12] table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_type=136,icmp_code=0,nd_target=fe80::f816:3eff:fece:bd06,nd_ttl=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12]
subtable6	0x906aa52e	table=74, priority=80,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_t

		ype=136 actions=load:0x1->NXM_NX_REG10[12] table=74, priority=80,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_t ype=136 actions=load:0x1->NXM_NX_REG10[12] table=74, priority=80,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_t ype=135 actions=load:0->NXM_NX_REG10[12] table=74, priority=80,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_t ype=135 actions=load:0->NXM_NX_REG10[12]
subtable7	0x5d58d882	table=74, priority=80,arp,reg14=0x4,meta data=0x4 actions=load:0x1->NXM_NX_REG10[12] table=74, priority=80,arp,reg14=0x5,meta data=0x4 actions=load:0x1->NXM_NX_REG10[12]

- 查找subtable时，通过计算flow中的mask的hash值，在 `cls->subtables_map` 中查找的。
- 但是在查找rule时，是遍历 `cls->subtables` 变量，这个 `cls->subtables` 是以优先级的从高到低的顺序来保存的subtable。

subtable→rules的插入流表

对于rule，则以所有分段 `flow & mask` 的累计hash值作为key，rule为value来保存到subtable→rules中。

比如subtable1的rules如下：

rule名称（假设的）	Hash 值	OpenFlow 流表规则
rule1	0x6e3e6576	table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_t ype=135,icmp_code=0,nd_sll=f a:16:3e:ce:bd:06 actions=load:0->NXM_NX_REG10[12]

rule2	0x19de917d	table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12]
rule3	0x1bd15ebd	table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=fa:16:3e:14:72:eb actions=load:0->NXM_NX_REG10[12]
rule4	0xcd376290	table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=00:00:00:00:00:00 actions=load:0->NXM_NX_REG10[12]

```

1
2 subtable1->rules[0x6e3e6576] = {
3     table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_
4 }
5
6 subtable1->rules[0x19de917d] = {
7     table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_
8 }
9
10 subtable1->rules[0x1bd15ebd] = {
11     table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_
12 }
13
14 subtable1->rules[0xcd376290] = {
15     table=74, priority=90,icmp6,reg14=0x4,metadata=0x4,nw_ttl=255,icmp_
16 }

```

subtable中的rules的同一个hash key中，有可能会保存多条rule，按照优先级排序保存的。

subtable→indices的构造

在 `insert_subtable` 函数中，`subtable->index_maps` 保存有效段mask(比如L2的flowmap不为空则为有效段mask)的stage_map (即flowmap)。`subtable->n_indices` 保存的有效段mask的index(等于有效段mask个数-1)。

在 `classifier_replace` 函数中，根据 `subtable->index_maps` 中的数据和对应的match数据，计算有效段的累计hash值 (`match & mask` 的计算)。其中除最后一个有效段的累计hash值，前面的有效段的累计hash值都

保存到 `subtable->indices` 中，最后一个有效段的累计hash值作为key，设置到 `subtable->rules` 中，用于查询。

比如openflow的metadata，L2和L4段有值，则这3个段为有效段。

- `subtable->index_maps[0]` 保存metadata段mask的flowmap，`subtable->index_maps[1]` 保存L2段mask的flowmap，`subtable->index_maps[2]` 保存L4段mask的flowmap。
- `subtable->n_indices` 的值为2。
- `subtable->indices[0]` 保存metadata段 `match & mask` 的计算的hash值，`subtable->indices[1]` 保存metadata段+L2段 `match & mask` 的计算的累计hash值，而metadata段+L2段+L4段 `match & mask` 的计算的累计hash值作为key，设置到 `subtable->rules` 中。

这里我们以 `table=74`,

```
priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_type=135,icmp_code=0,nd_sll=fa:16:3e:ce:bd:06 actions=load:0->NXM_NX_REG10[12]
```

 openflow举例说明，通过查看上面的 `struct flow` 来进行分段计算hash值。

这里包括metadata，L2,L3和L4段（这里L2虽然没有显示字段，但是L2中的 `dl_type` 会默认设置的），最终设置的数据如下：

- `subtable->n_indices = 3`
- `subtable->index_maps[0]` 保存metadata段mask的flowmap，`subtable->index_maps[1]` 保存L2段mask的flowmap，`subtable->index_maps[2]` 保存L3段mask的flowmap，`subtable->index_maps[3]` 保存L4段mask的flowmap。
- `subtable->indices[0] = 0x3227a870`, `subtable->indices[1] = 0xf59fb913`, `subtable->indices[2] = 0xe997f639`
- 其中metadata段+L2段+L3段+L4段 `match & mask` 的计算的累计hash值为0x6e3e6576，作为key设置到 `subtable->rules` 中。

```
1 subtable1->rules[0x6e3e6576] = {
2     table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,nw_ttl=255,icmp_t
3 }
```

openflow流表插入到classifier的流程

当插入一条openflow到classifier时，步骤如下：

- 在 `find_subtable` 函数中，根据这条openflow的mask的hash值，在 `cls->subtables_map` 中查找对应的subtable。
- 如果没有找到对应的subtable，则创建一个subtable，并且以mask的hash值作为key设置到 `cls->subtables_map` 中。
- 在创建subtable时，会将该openflow进行有效分段，将其有效分段的 `flow & mask` 的累计hash值保存到 `subtable->indices` 中，用于快速查询。
- 将该openflow所有有效分段的 `flow & mask` 的累计hash值作为key设置到 `subtable->rules` 中。
- 在最后将创建的subtable通过优先级排序设置到 `cls->subtables` 中。

table 74的规则查找

我们这里flow如下：

```
1 in_port=1,eth_src=fa:16:3e:ce:bd:06,eth_dst=fa:16:3e:88:32:5c,dl_type=0x
```

通过以上的flow去classifier查找到rule，设置对应的wc，即mask。

这里我们主要关注到table 74中的rule的查找。通过分析和debug可以知道nd_target和nd_tll的设置是在查询了table 74的subtable2后设置的。

通过table 74的subtable1的查询后，megaflow如下：

```
1 recirc_id=0,eth,icmp6,reg10=0/0x1000,reg11=0xb,reg12=0xa,reg13=0xd,reg14
```

subtable2的规则如下：

```
1
2 subtable2->rules[0x7f849edb] = {
3     table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:c
4 }
5
6 subtable2->rules[0x96fccc3] = {
7     table=74, priority=90,icmp6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:c
8 }
```

这里我们进行分段：

分段名称	openflow（经过subtable1后的）	保存的subtable2->rules[0x7f849edb]（这里就查询到第一条就截至了，查找下一个subtable3了）
metadata	reg10=0/0x1000,reg11=0xb,reg12=0xa,reg13=0xd,reg14=0x5,metadata=0x4,in_port=1	reg14=0x5,metadata=0x4
L2	eth,dl_src=fa:16:3e:ce:bd:06	eth(默认会设置上去),dl_src=fa:16:3e:ce:bd:06
L3	icmp6,ipv6_src=fd1f:9e62:3182:608c::a2,ipv6_dst=fd1f::/16,nw_ttl=0,nw_frag=no	icmp6
L4	NULL	icmp_type=136,icmp_code=0,nd_target=fd1f:9e62:3182:608c::a2,nd_tll=00:00:00:00:00:00

这里经过subtable1后的openflow与subtable2→index_maps（这里保存的mask）分段进行hash后，到对应分段的subtable2→indices的进行查询，看是否有匹配的。更直观点：只要在subtable2→rules的分段部分分别被包括到对应的经过subtable1后的openflow中，那么就会匹配到。即：

```
1 flow & mask如下：
2 0 & 1 = 0
3 1 & 1 = 1
4 1 & 0 = 0
5 0 & 0 = 0
```

只要满足 $1 \& 1 = 1$ 和 $1 \& 0 = 0$ 两种情况，就会匹配到。

我们观察到L4属于 $0 \& 1 = 0$ 的情况，这种就无法匹配到。所以直接查询下一个subtale3。就是这里无法匹配到的同时，classifier还把这个L4分段的mask设置到wc中了。所以经过subtale2后，生成的megaflow如下：

```
1 recirc_id=0,eth,icmp6,reg10=0/0x1000,reg11=0xb,reg12=0xa,reg13=0xd,reg14
```

这里就把 `icmp_type`，`nd_target` 和 `nd_ttl` 字段设置到wc中了。

当最后一个stage(L4分段)无法匹配，还是会将该分段的mask设置到wc中。其原因是社区合入了这个patch：[🔗 classifier:](#)

[Fix missing masks on a final stage with ports trie. · openvswitch/ovs@489553b](#)